Understanding graph representation learning in reinforcement learning based logic optimization

Anonymous Author(s)
Affiliation
Address
email

Abstract

Computer chips ought to be as small as possible for cost efficiency. The problem of minimizing chip size while preserving the chip's functionality has been studied through the prism of and-inverter graphs (AIGs) since any boolean function can be represented by AIGs. Because the complexity of AIG size minimization is unknown—it is an instance of the Minimum Circuit Size Problem, which lies in NP but is not known to be NP-hard—it is a fitting candidate for neural solvers that can leverage learned representations to generalize to unseen AIGs and outpace standard enumerations and branch-and-bound solvers. Reinforcement learning and more generally planning have been used to optimize such AIGs without the need for intractable supervision signals. Contemporarily, graph representation learning have been applied to train models that can perform tasks on unseen AIGs. More recently, those two technologies have been combined and achieved competitive performances with state-of-the-art heuristics. In this preliminary work, we question the adequacy of graph representation learning to improve deep reinforcement learning for logic AIG minimization.

1 Introduction

A circuit that computes a given Boolean function with fewer gates occupies less area and is cheaper to manufacture, which matters at the scale of the semiconductor industry [29]. Logic synthesis addresses this at the technology-independent level: given a Boolean function, find a small circuit that computes it. And-Inverter Graphs (AIGs) are the representation used for this step, because AND and NOT are functionally complete—so every Boolean function admits an AIG.

AIG minimization has no known exact algorithm that scales. It is an instance of the Minimum Circuit Size Problem, which lies in NP but is not known to be NP-hard [15], and which is related to Graph Isomorphism [2]. In practice, AIGs are reduced by applying sequences of local rewriting heuristics such as those implemented in `ABC` [4]. Choosing the sequence is itself a search problem, and it has been addressed with Bayesian optimization [10], multi-armed bandits [28], reinforcement learning [14, 48] and Monte Carlo tree search [31].

Several of these methods encode the current AIG with a graph neural network rather than with a fixed set of summary statistics [48, 31, 47]. The motivation is the same as the very first deep reinforcement learning algorithm, which was designed specifically for Backgammon [39]: the state space for an AIG minimization is huge and cannot be enumerated. Most importantly, learned representation of the circuit should carry structural information that statistics discard, and that it is what should allow a deep reinforcement algorithm [34] to train efficiently by generalizing to unseen circuits: "I have seen a similar graph before, I should probably edit the AIG in a similar way to get good results". We are not aware of a study that isolates the contribution of the AIG encoder in the reinforcement learning training. This work reports such an ablation.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

Contributions: First, we show that when replicating past work's graph reinforcement learning pipelines, graph representation learning did not help the training compared to simple multi-layer perceptrons trained with graph statistics (number of nodes, edges, ...). More fundamentally, we show that both theoretically and empirically many graph representations, even AIG-specific ones, cannot predict the values of AIGs–in the RL sense–more accurately than standard multi-layer perceptrons trained on graph statistics only.

2 Related work

2.1 Combinational optimization

Combinational synthesis is a class of fast heuristics that reduce an AIG by rewriting it locally, and that work well in practice [22, 23, 6]. `ABC` [4] is the reference implementation and is widely used in academia; its standard primitives are `balance`, `rewrite`, `resub` and `refactor`. For instance, `refactor` performs iterative collapsing and refactoring of logic cones in the AIG, attempting to reduce both the number of nodes and the number of levels. Since no primitive is globally optimal, reduction is done by composing them into sequences. Hand-designed sequences are the reference points against which learned methods are measured: `resyn` [30] applies `balance`, `rewrite`, `balance`, `rewrite`, `balance`, and the `ABC` documentation also ships a more aggressive sequence known as the "lazy man's synthesis" [45]. Searching over sequences has been automated without learned circuit representations, with Bayesian optimization [10] and with multi-armed bandits [28].

2.2 Graph representation learning for AIGs

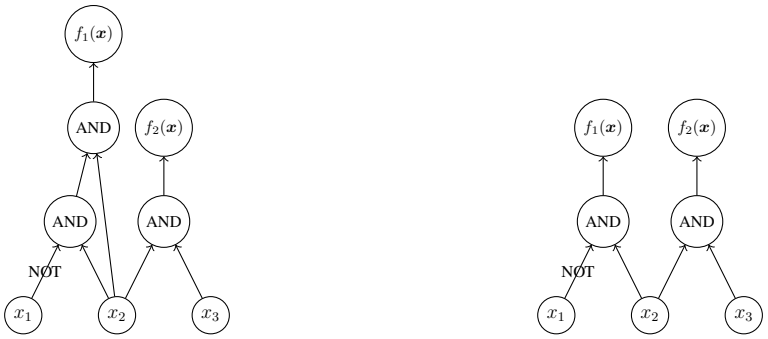
A separate line of work learns representations of circuits that are meant to transfer across circuits, rather than a policy for one circuit. DeepGate4 [47] is representative: it learns node embeddings of AIGs and scales them to large designs, and is one of several AIG-specific encoders that we evaluate in Section 5.2. `GraphBench` [36] is, to our knowledge, the first AIG reduction benchmark built for methods that must perform on unseen AIGs. It provides a training set of Boolean functions of varying input and output dimension and a disjoint test set, and it reports node reduction relative to the expert `ABC` sequence `resyn2`, `resyn2`, `dc2`, `resyn2rs`, `resyn2`. Its main result is negative: given the AIG obtained by the naive construction of Section 3.1, specialised DAG generative models [5, 19] do not preserve functional equivalence while reducing size. Our results are negative in the same sense, for a different family of models.

2.3 Reinforcement learning for AIGs

Hosny et al. [14] formulate synthesis as an MDP whose states are vectors of AIG statistics—numbers of primary inputs and outputs, nodes, edges, levels and latches, and the proportions of AND and NOT nodes—whose actions are `ABC` primitives, and whose reward combines node count and level count. Zhu et al. [48] keep `ABC` primitives as actions but encode the whole AIG with a graph convolutional network, alongside a small vector of statistics and recent actions, and reward the relative reduction in node count. This is the formulation we adopt, and we state it in full in Section 4.1. Haaswijk et al. [11] predate both and act with Boolean algebraic operations on majority-inverter graphs; we do not compare against it directly because MIGs and AIGs are not directly comparable. Both Hosny et al. [14] and Zhu et al. [48] train with policy gradient methods [38, 43, 26]. We are not aware of a value-based [42, 25] treatment, even though AIG editing is deterministic. These methods report reductions beyond the `ABC` sequences, but they are not designed to generalise: a policy is trained to reduce one AIG, so the training cost is paid again for every new circuit. Search-based methods lift that restriction. AlphaSyn [31] and AlphaChip [21] plan online and therefore apply to circuits unseen at training time, but there is no separation between training and inference: every edit requires a lookahead, and every lookahead step requires network forward passes.

2.4 Graph reinforcement learning for other combinatorial optimization

AIG reduction is an instance of graph structure optimization in the taxonomy of Darvariu et al. [7], who survey reinforcement learning applied to combinatorial problems on graphs and report that a



$\mathcal{G}_1$: 8 nodes, 4 levels. $\mathcal{G}_2$: 7 nodes, 3 levels.

Figure 1: Two functionally equivalent AIGs with different numbers of nodes for the Boolean function $f(x_1, x_2, x_3) = (\bar{x_1} \wedge x_2, x_2 \wedge x_3)$.

graph neural network encoder is the common choice of state representation. Within chip design the same combination is used for transistor sizing [41] and for macro placement [21]; in both cases the encoder is motivated as the component that enables transfer to new circuits.

3 Background material

3.1 And-inverter graphs minimization

Given an arbitrary Boolean function $f : \{0,1\}^n \to \{0,1\}^m$, the objective is to find an And-Inverter Graph that encodes f with as few nodes as possible.

Definition 1 (And-Inverter Graph, following section 3.2.1 from Stoll et al. [36]) *An And-Inverter Graph $\mathcal{G} := (V, E)$ is a directed acyclic graph of in-degree (fanin) at most two. The node set is partitioned into input nodes of in-degree 0, $V^{in} := \{v_1^{in}, \ldots, v_n^{in}\}$, AND nodes of in-degree two, $V^{AND} := \{v_1^{AND}, \ldots, v_k^{AND}\}$, and output nodes of in-degree one, $V^{out} := \{v_1^{out}, \ldots, v_m^{out}\}$; AIGs are therefore heterogeneous graphs [35]. The edge set is $E \subseteq V \times V \times \{0,1\}$, with 0 indicating direct transmission and 1 indicating input negation (NOT). For an input $\boldsymbol{x} \in \{0,1\}^n$, values are assigned to input nodes, propagated through gate nodes (applying inversion where specified), and collected at output nodes to yield $\mathcal{G}(\boldsymbol{x}) \in \{0,1\}^m$. Because the combination of AND and NOT is functionally complete, any Boolean function can be represented using only these two operations.*

We write $\mathcal{G}_f := \{\mathcal{G} : \mathcal{G}(\boldsymbol{x}) = f(\boldsymbol{x}) \; \forall \boldsymbol{x} \in \{0,1\}^n\}$ for the set of AIGs functionally equivalent to f. AIGs are non-canonical representations of Boolean functions [24]—two different AIGs can represent the same function—so $|\mathcal{G}_f| > 1$ in general, as in Figure 1.

Definition 2 (And-Inverter Graph reduction) *Given a Boolean function $f : \{0,1\}^n \to \{0,1\}^m$, we want to find*

$$\mathcal{G}^\star \subseteq \arg\min_{\mathcal{G} \in \mathcal{G}_f} |V|,$$

the AIGs functionally equivalent to f with as few nodes as possible.

The hardness of this problem is not known. It is an instance of the Minimum Circuit Size Problem [15], a natural and well-studied problem in NP that is not known to be NP-hard[1], and which is related to Graph Isomorphism [2]. In the absence of a breakthrough in complexity theory, this motivates the development of heuristics.

Obtaining an AIG. An AIG for f is obtained from its truth table $\{0,1\}^{2^n \times (m+n)}$ by writing f as a disjunctive normal form—for each output and each of the 2^n rows, a conjunction of n literals—which takes $O(mn2^n)$ time and yields at most $mn2^n$ literals. The DNF is then factored with De Morgan's laws so that it uses only ANDs and NOTs, and one AND node is added to the AIG per AND symbol

[1] `https://mcsp.work/index.html`

in the factored form. While doing so, one-level structural hashing (`strashing`) [24] adds a node only if no node with the same inputs already exists, which requires storing a dictionary from input pairs (and their permutation) to node indices. The resulting AIG has at most $mn2^n - 1$ AND nodes, n input nodes and m output nodes. This is the starting point that combinational synthesis then reduces.

3.2 Markov decision process

AIG reduction can be formulated as a sequential decision making task in which the environment is the current AIG, the actions are edits of that AIG, and the rewards are shaped to favour edits that remove nodes. Formally, this is the problem of finding an optimal policy for a Markov decision process [3, 32].

Definition 3 (Markov decision process) *An MDP is a tuple $\mathcal{M} = \langle S, A, R, T, T_0 \rangle$. S is a finite set of states representing all possible configurations of the environment (e.g. AIGs, or AIG statistics). A is a finite set of actions available to the agent (e.g. AIG edits). $R : S \times A \to \mathbb{R}$ is a deterministic reward function that assigns a real-valued reward to each state-action pair (e.g. number of deleted nodes). $T : S \times A \to \Delta(S)$ is the transition function that maps state-action pairs to probability distributions over next states $\Delta(S)$ (e.g. how graph edits change the AIG; here the transitions are deterministic). $T_0 \in \Delta(S)$ is the initial distribution over states (e.g. a set of AIGs to reduce).*

Definition 4 (Reinforcement learning objective) *Given an MDP $\mathcal{M} \equiv \langle S, A, R, T, T_0 \rangle$, the goal is to find a policy $\pi : S \to A$ that maximises the expected discounted sum of rewards*

$$J(\pi) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, \pi(s_t)) \mid s_0 \sim T_0, s_{t+1} \sim T(s_t, \pi(s_t))\right]$$

where $0 < \gamma \leq 1$ is the discount factor that controls the trade-off between immediate and future rewards.

We write $V^\star(s)$ for the value of a state under an optimal policy, i.e. the discounted sum of rewards collected from s by acting optimally. $V^\star$ is the quantity a critic is trained to approximate in actor-critic methods, and the quantity a state representation must be able to distinguish states by; Section 5.2 uses it to measure what a representation can express, independently of any training procedure. Reinforcement learning algorithms optimise the objective of Definition 4 by trial and error; we refer to Sutton and Barto [37] for an introduction.

4 Methodology

We keep the reinforcement learning pipeline of past work fixed and vary only how the current AIG is presented to the policy. The MDP, the policy and value heads, the training algorithm and the hyper-parameter grid are identical across the runs we compare, so the encoder is the only difference between two of them.

4.1 The Markov decision process

We use the formulation of Zhu et al. [48], instantiating Definition 3 as follows. Let $\mathcal{G}_0$ be the AIG to reduce, and write n_t and ℓ_t for the number of AND nodes and the number of levels of the AIG after t actions.

Actions. A is a set of five `ABC` primitives,

$$A = \{\texttt{balance}, \texttt{rewrite}, \texttt{rewrite -z}, \texttt{refactor}, \texttt{refactor -z}\},$$

where `-z` enables zero-cost replacements. This is the action set of Zhu et al. [48]; our environment also implements a seven-action variant adding `resub` and `resub -z`, and the full `ABC` action space, neither of which is used in the experiments below.

Transitions and initial state. T is deterministic: the selected primitive is run on the current AIG. Every primitive preserves functional equivalence, so every reachable state lies in $\mathcal{G}_f$ and the problem of Definition 2 is never violated, only under-solved. T_0 is a point mass on $\mathcal{G}_0$, read from the benchmark file at the start of each episode.

Horizon. Episodes last exactly $H = 20$ actions and terminate at $t = H$, with no early stopping. This makes the induced optimization a search over flows of length 20 from a five-element alphabet which is again the exact setting of Zhu et al. [48].

Reward. The reward is the relative reduction in AND count with respect to the initial AIG,

$$R(s_t, a_t) = 1 - \frac{n_{t+1}}{n_0},$$

and is issued at every step rather than only at the end. It is a level and not a difference: with $\gamma = 1$ the return $\sum_t (1 - n_t/n_0)$ is maximised by an agent that keeps the AIG small throughout the episode, not merely by one that ends small. Levels do not enter the reward; unlike the multi-objective reward of Hosny et al. [14], we optimise node count alone, which is the objective of Definition 2. We report the smallest AND count seen during an episode, $\min_{t \le H} n_t$.

Observations. The agent does not see the state directly but through an observation with two parts, a graph and a vector, described in Section 4.2. The vector part is always present; the graph part is what the encoders under comparison do or do not consume.

4.2 Graph state representation

Message passing neural networks. Every graph encoder considered in this paper belongs to the message passing family [9]. A message passing neural network attaches a vector $h_v^{(0)}$ to each node v, initialised from that node's features, and refines it over k rounds:

$$h_v^{(t+1)} = \phi^{(t)}\Big(h_v^{(t)}, \bigoplus\nolimits_{u \in \mathcal{N}(v)} \psi^{(t)}\big(h_v^{(t)}, h_u^{(t)}, e_{uv}\big)\Big),$$

where $\mathcal{N}(v)$ are the neighbours of v, e_{uv} is an optional label on the edge between u and v, $\bigoplus$ is a permutation-invariant aggregator, and $\phi^{(t)}, \psi^{(t)}$ are learned. A graph-level vector is then obtained by pooling $\{h_v^{(k)}\}_{v \in V}$ with a second permutation-invariant readout. Architectures in this family differ only in the choice of ψ, $\bigoplus$ and ϕ: GCN [17], GIN [44], GAT [40] and GraphSAGE [13] are instances, and so are the AIG-specific encoders we evaluate in Section 5.2.3. They also share a limitation. After k rounds, two nodes that k iterations of the 1-dimensional Weisfeiler-Leman refinement assign the same colour carry the same embedding [44, 27], so no network in the family can tell apart two AIGs that 1-WL cannot. Section 5.2.2 turns this into a lower bound on the error of every such encoder.

4.2.1 Engineered graph features

The vector part of the observation, which we write $x_{\mathcal{G}} \in \mathbb{R}^{|A|+5}$, holds a summary of the current AIG and of the recent history of the episode:

$$x_{\mathcal{G}} = \Big[\underbrace{\tfrac{1}{q}\textstyle\sum_{j=1}^{q} \mathbf{1}[a_{t-j} = a]}_{a \in A},\ \ \frac{n_t}{n_0},\ \frac{\ell_t}{\ell_0},\ \frac{n_{t-1}}{n_0},\ \frac{\ell_{t-1}}{\ell_0},\ \frac{t}{H}\Big].$$

The first $|A| = 5$ entries are a normalised histogram of the last q actions; we set $q = H$, so they summarise the whole flow applied so far. The next four are the AND count and the level count of the current and of the previous AIG, each divided by its value in $\mathcal{G}_0$, so that the vector is scale-free and comparable across benchmarks of very different sizes. The last entry is the normalised time step, which makes the finite-horizon problem Markov in the observation. Note that $x_{\mathcal{G}}$ carries no structural information about the circuit: two AIGs with the same size, depth and history are indistinguishable to it. It is computed in a single pass over the graph and adds no parameters of its own.

4.2.2 Graph convolutional networks

The graph part of the observation is the AIG itself, as in Zhu et al. [48]: nodes carry a one-hot encoding of their type among the six types `abc_py` exposes, and a directed edge is added from each fanin to the node it feeds. Edges carry no label, so complementation is visible to the encoder only through node types. A graph convolutional network [17] performs a fixed number of message passing rounds over this graph and pools the resulting node embeddings into a graph-level vector, which is concatenated with $x_{\mathcal{G}}$ before the policy and value heads. We instantiate four such encoders, listed in Table 1, spanning $2{,}496$ to $107{,}968$ parameters added to the policy. The three smaller ones hold the

Table 1: The four graph encoders. Depth is the number of message passing rounds, width the hidden dimension, and output the dimension of the pooled vector concatenated with $x_{\mathcal{G}}$. The baseline uses no encoder at all.

encoder	depth	width	output
`small`	3	12	4
`medium`	3	64	32
`large`	3	128	64
`v. large`	5	128	64

depth fixed at three rounds and vary the width; the largest also raises the depth to five, so that depth and width are not confounded.

The baseline is the same architecture with the graph part of the observation dropped, so that the policy reads $x_{\mathcal{G}}$ alone through a multi-layer perceptron. The comparison is therefore not between two competing representations but between $x_{\mathcal{G}}$ and $x_{\mathcal{G}}$ augmented with a learned representation of the circuit: the graph encoder is only ever added to the baseline, never substituted for it.

4.3 Reinforcement learning

Policies are trained with PPO [34] on the MDP of Section 4.1. Algorithm 1 states the resulting procedure, which we call GRAPHPPO. It is ordinary PPO with the encoder made explicit: line 7 is the only place where the five configurations we compare differ, and the encoder is trained end to end with the policy and value heads rather than pre-trained or frozen.

We use the implementation of `stable-baselines3` [33] with $\gamma = 1$: the horizon is finite and fixed, so the return is bounded without discounting and there is no reason to prefer early reductions over late ones. Eight environments run in parallel and each contributes 20 steps per update, so an update is computed from 160 transitions and a single episode spans five updates. Training lasts $8{,}000$ environment steps, that is $N = 50$ updates. The policy and value heads are each two hidden layers of 256 units and share the encoder of line 7. After every update we evaluate the greedy policy for one episode, which is the quantity the learning curves report. Because the comparison is between encoders and not between tuned systems, each encoder is run over the same grid of learning rates, entropy coefficients and seeds, and encoders are compared only on the grid cells that all of them completed.

5 Experiments

5.1 Graph reinforcement learning

Setup. The benchmarks are the ten MLCAD circuits `apex1`, `bc0`, `C1355`, `C5315`, `C6288`, `C7552`, `dalu`, `i10`, `k2` and `mainpla`. Each of the five encoders is run on each benchmark at every combination of three learning rates (10^{-4}, 10^{-5}, 10^{-6}), three entropy coefficients (0, 0.01, 0.1) and five seeds, for $2{,}250$ runs in total. Twenty-six of them (eighteen `small`, eight `v. large`) stopped before the fiftieth update; we drop them rather than pad them, and requiring all five encoders to have completed a given (benchmark, learning rate, entropy, seed) cell leaves 424 of the 450 cells. Every number below averages over that same set of cells, so no encoder is given its own best hyper-parameters — tuning each separately would let the selection step, rather than the encoder, decide the winner. We report the final AND count divided by the AND count of the input AIG, so that benchmarks of different sizes can be averaged.

Adding a GCN to the policy does not improve AIG minimization (Figures 2, 3). Every GCN reduces the final AND count by 0.76–0.85% relative to the MLP, which is less than the 1.23% standard deviation induced by changing the seed alone, and a $43\times$ increase in encoder size moves that figure by nothing.

The result holds over a hundred further circuits and does not grow with circuit size (Figures 4, 5). We repeat the comparison on `ex200`–`ex299`, a suite spanning 106 to $46{,}977$ input AND gates. The

Algorithm 1 GRAPHPPO. The encoder f_θ is the only component that varies across the runs we compare; everything else is held fixed.

Require: AIG $\mathcal{G}_0$ to reduce with n_0 AND nodes, encoder f_θ, policy head π_θ, value head V_θ, action set A of five `ABC` primitives, horizon $H = 100$, iterations N, epochs K, clipping ϵ, entropy weight β, value weight c, step size η

$n^\star \leftarrow n_0$
for $i = 1, \dots, N$ **do**
  $\mathcal{D} \leftarrow \emptyset$
  for each parallel environment **do**
    $\mathcal{G} \leftarrow \mathcal{G}_0$
    for $t = 0, \dots, H-1$ **do**
      $s_t \leftarrow \big(f_\theta(\mathcal{G}),\ x_{\mathcal{G}}\big)$ ▷ f_θ is a GCN, or is absent for the baseline
      $a_t \sim \pi_\theta(\cdot \mid s_t), \quad a_t \in A$
      $\mathcal{G} \leftarrow \textsc{Abc}(\mathcal{G}, a_t)$ ▷ deterministic, preserves functional equivalence
      $r_t \leftarrow 1 - |V^{AND}(\mathcal{G})| \,/\, n_0$ ▷ relative size, issued at every step
      $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s_t, a_t, r_t, \log \pi_\theta(a_t \mid s_t), V_\theta(s_t))\}$
      $n^\star \leftarrow \min\big(n^\star,\ |V^{AND}(\mathcal{G})|\big)$
    end for
  end for
  compute returns $\hat{R}_t$ and advantages $\hat{A}_t$ from $\mathcal{D}$
  $\theta_{\text{old}} \leftarrow \theta$
  for K epochs over minibatches of $\mathcal{D}$ **do**
    $\rho_t(\theta) \leftarrow \pi_\theta(a_t \mid s_t) \,/\, \pi_{\theta_{\text{old}}}(a_t \mid s_t)$
    $L(\theta) \leftarrow \widehat{\mathbb{E}}_t\big[\min\big(\rho_t \hat{A}_t,\ \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\hat{A}_t\big) - c\big(V_\theta(s_t) - \hat{R}_t\big)^2 + \beta\, \mathcal{H}\big[\pi_\theta(\cdot \mid s_t)\big]\big]$
    $\theta \leftarrow \theta + \eta\, \nabla_\theta L(\theta)$ ▷ gradients flow through f_θ
  end for
end for
return $n^\star$, the smallest AND count visited

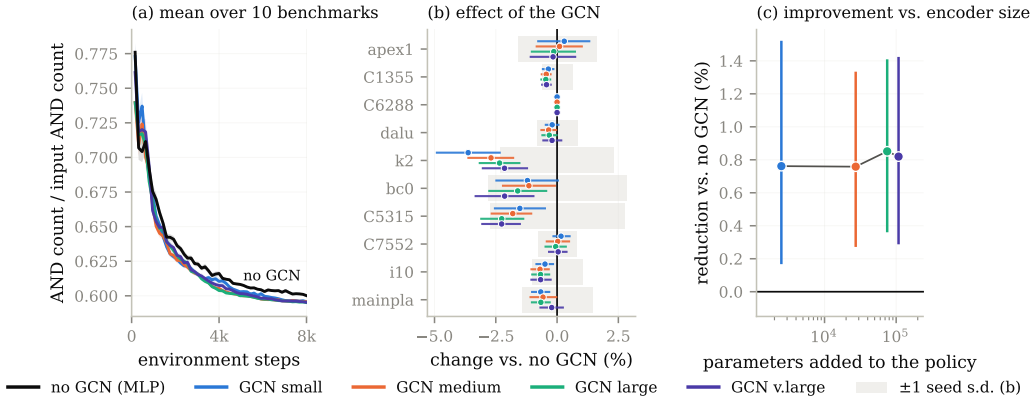


Figure 2: PPO on the ten MLCAD benchmarks with five observation encoders, hyper-parameter matched. (a) mean AND count over the ten benchmarks; (b) per-benchmark change against the MLP, shaded band is one seed standard deviation; (c) mean reduction against the parameters the encoder adds to the policy.

three GCNs reduce the final AND count by 0.33–0.71% against a reseed standard deviation of 0.54%, and the effect is flat across two and a half orders of magnitude of circuit size. We note that the MLP runs were executed on CPU and did not finish on the largest circuits, which is why 20 benchmarks are excluded.

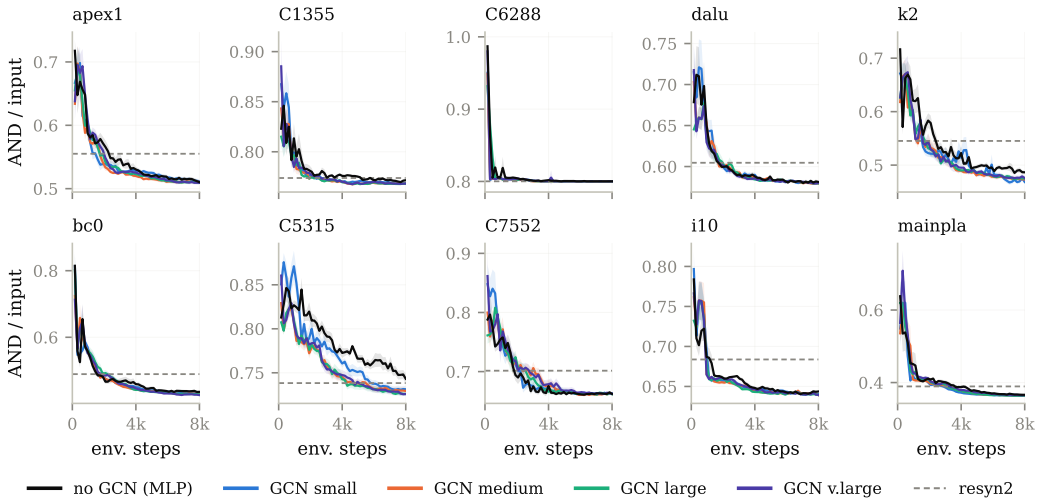


Figure 3: Per-benchmark learning curves for the runs aggregated in Figure 2(a). Dashed line: `resyn2`.

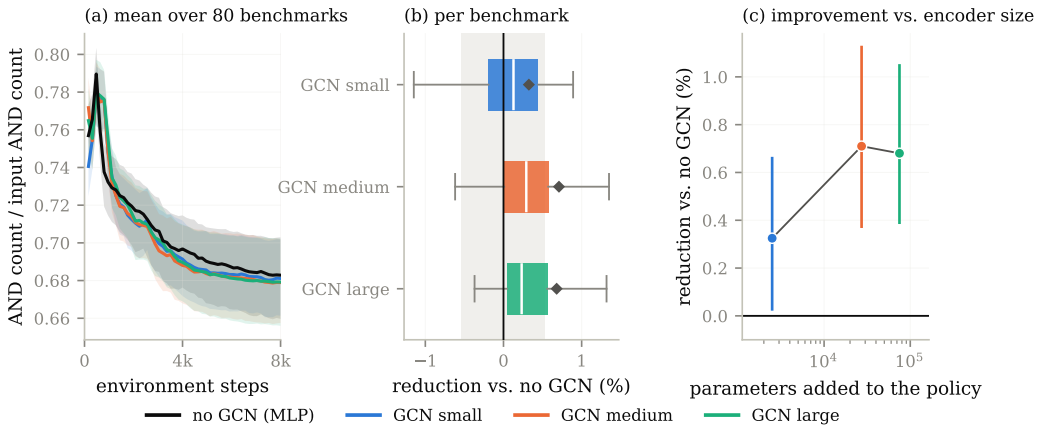


Figure 4: The same comparison on `ex200`–`ex299`, where each encoder ran at its own baseline hyper-parameters. Restricted to the 390 of 500 (benchmark, seed) cells all four encoders completed, covering 80 benchmarks.

5.2 Value prediction

The previous section compares encoders inside one algorithm, so a null result there could be a property of that algorithm rather than of the representations. We therefore ask a question that does not involve a training algorithm at all: how well can the optimal value $V^\star$ of a state be predicted from what a given class of models is able to see?

5.2.1 Building a dataset

Computing $V^\star$ exactly is out of reach: the tree of flows over the five primitives of Section 4.1 has 5^H leaves, and every edge of it costs one `ABC` call on the full circuit. We therefore build, for each benchmark, a dataset of states paired with a Monte-Carlo estimate of their value, obtained by sampling both the states and the continuations from them.

States. A state is the AIG obtained by applying a uniformly random flow to the benchmark $\mathcal{G}_0$: we draw a length $L \sim \mathcal{U}\{2, \ldots, 10\}$, then L primitives independently and uniformly from the five-element action set, and run them in order. We sample 1,000 such states per benchmark. Each one is written to disk as an `.aig` file and stored together with the statistics $x_{\mathcal{G}}$ of Section 4.2.1 recorded

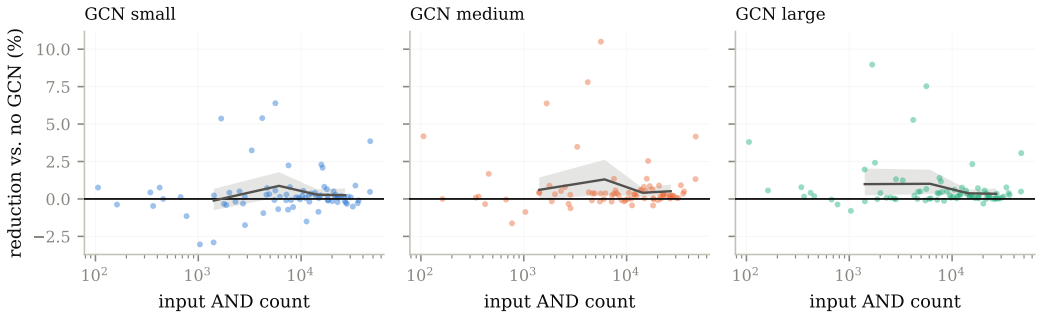


Figure 5: Reduction against the MLP as a function of input circuit size on `ex200`–`ex299`. Line: mean over size quartiles.

at that point of the flow, so that every model class of Sections 5.2.2 and 5.2.3 is evaluated on exactly the same states and the same targets—the graph models read the `.aig` file, the MLP reads $x_{\mathcal{G}}$.

Targets. From each stored state s we run $M = 100$ independent rollouts of 8 further uniformly random primitives, reload the state between rollouts, and record the AND count $n^{(j)}(s)$ reached at the end of rollout j. The target is the best of them,

$$\widehat{V}(s) = \min_{j \le M} n^{(j)}(s),$$

that is, the smallest AND count the sampler managed to reach within 8 actions of s. This is a best-of-M estimate: it is an upper bound on min over the 5^8 flows of length eight, and it approaches it from above as M grows. Producing one dataset therefore costs $1{,}000 \times (L + M \times 8) \approx 8 \times 10^5$ `ABC` primitive calls per benchmark and per seed, which is why M is lowered to 10 on `C6288`, by far the most expensive circuit of the suite to rewrite.

What the target measures. $\widehat{V}$ is expressed in AND nodes rather than in returns, so it is ordered the other way round from the value of Definition 4—smaller is better—and it truncates the horizon at eight steps instead of the $H = 20$ of the reinforcement learning experiments. Neither affects the comparison between model classes. The map $n \mapsto 1 - n/n_0$ from AND count to the reward of Section 4.1 is affine and order-reversing, all the errors we report are ratios to the variance of the target and are therefore free of its units, and the rank correlations we report are unchanged up to sign. What the truncation does mean is that our targets approximate the value of an eight-step-to-go problem under a best-of-M estimate, not the exact $V^\star$ of the full episode, and the bounds of Section 5.2.2 are lower bounds on the error of predicting *that* quantity.

Protocol. The procedure is repeated with ten seeds (0 to 9) per benchmark, each seed drawing its own 1,000 states and its own rollouts, and each resulting dataset is split at random into training, validation and test sets; the figures of this section report over those ten repetitions. The benchmarks are the same ten MLCAD circuits as in Section 5.1, and results are reported over nine of them. Two circuits are degenerate for this purpose: on `C1355` the sampler reaches only six distinct values of $\widehat{V}$, and on `C6288` only two, so the latter is excluded where noted.

5.2.2 Theoretical lower bounds on the prediction error

A model that cannot distinguish two states must assign them the same prediction. For a class of models, this partitions the state space, and the best achievable squared error is attained by the predictor that outputs, for each part, the mean target over that part. That error is a lower bound: no model in the class can beat it, regardless of its size, its training procedure or its optimisation. This is the protocol of `Graphtester` [1], which measures the theoretical boundary of graph neural networks on a given dataset rather than the performance of one trained instance; we reimplement it so that it applies to our four state representations and to the target of Section 5.2.1.

The classes we bound. We compute the bound for four classes, ordered by what they can see. A *size* model sees the number of nodes $|V_{\mathcal{G}}|$ alone—this is what a 0-layer message passing network reduces to, since with no rounds of message passing the readout can only count nodes. A *size and depth* model sees the pair $(|V_{\mathcal{G}}|, d_{\mathcal{G}})$, where $d_{\mathcal{G}}$ is the logic depth reported by `ABC`. An $x_{\mathcal{G}}$ model sees the engineered statistics of Section 4.2.1, which is what the MLP baseline reads. A *message passing* model sees the 1-WL colouring of the AIG, which upper bounds the separating power of every architecture of Section 4.2 by the argument given there. Comparing the third and the fourth is the question of this section: what a learned encoder could express, against what the statistics already express.

Computing the partitions. For the first two classes the partition is by exact equality of the descriptors. For $x_{\mathcal{G}}$, feature vectors are snapped to a grid of spacing 10^{-8} and grouped by equality of the snapped vector; we group on the five circuit and time entries only and discard the action histogram, so this bound is if anything conservative—the MLP of Section 4.3 sees strictly more than the model we bound here. For message passing, we group states by their 1-WL graph hash, computed with `networkx`'s [12] `weisfeiler_lehman_graph_hash` at every number of iterations $k = 1, \ldots, 50$, with each node labelled by its type.[2] We bound four views of the AIG, which is what the four message passing curves of Figures 6 and 7 report: the undirected graph $\mathcal{G}$, the directed graph $\mathcal{G}^{\rightarrow}$ in which refinement follows the fanin relation, and the two variants $\mathcal{G}^{e}, \mathcal{G}^{\rightarrow e}$ that additionally label each edge with its complement bit, so that inversion is visible to the refinement rather than only through node types. Errors are divided by $\max(1, \operatorname{Var} \widehat{V})$; in AND-node units the variance is orders of magnitude above one on every benchmark, so the clamp never binds and the reported quantity is the plain normalised error.

Metrics. Besides the squared error we report how well each class can *order* states, since a critic that ranks successors correctly is useful even when its absolute errors are large. We measure the rank association between $\widehat{V}$ and the group-mean predictor of the class with Spearman's ρ, Kendall's τ, and the weighted τ, which puts more weight on the top of the ranking—the states an agent would actually select. Rank correlations are computed at $k = 50$ refinement iterations.

A control for partition granularity. The bound is computed in sample: the group means are fitted on the same $1{,}000$ states the error is measured on. A partition fine enough to isolate every state therefore drives it to zero regardless of whether the representation carries any signal about $\widehat{V}$. We control for this by recomputing every bound with the targets randomly permuted across states—25 permutations per dataset, 8 on `mainpla` and `C6288`—which leaves the partition intact and destroys the association. The gap between a curve and its permuted counterpart, not the height of the curve, is what indicates that a representation carries information about the value; the same control is applied to the rank correlations, where the null is the correlation between $\widehat{V}$ and a random permutation of itself.

Message passing cannot predict $V^{\star}$ better than the statistics the MLP already sees (Figures 6, 7). A k-layer MPNN separates graphs at most as finely as k iterations of 1-WL, so grouping states by their 1-WL colour and predicting each group's mean $\widehat{V}$ gives an error no MPNN can beat; the same construction bounds a 0-layer network, a $(\text{size}, \text{depth})$ model, and a model seeing only the MLP's features $x_{\mathcal{G}}$. Message passing is worth a great deal over size and depth alone, but the MLP's own features close almost all of that gap: the two bounds are within a factor of 1.7 on eight of nine benchmarks and within 1.1 on five, and rank states equally well ($\tau \approx 0.98$). Depth is irrelevant — the bound moves by at most 1.1×10^{-3} between $k = 1$ and $k = 50$, and edge direction and inverter bits change it by less than 10^{-6}. `C6288` is excluded: its target takes two distinct values.

Limitations of the bounds. Four restrictions should be read with the numbers above. First, they hold over the state distribution of Section 5.2.1—AIGs reached by uniformly random flows of length two to ten—and not over the states a trained policy visits, which are fewer and more correlated; a representation could be uninformative on the former and useful on the latter. Second, each dataset descends from a single benchmark, so what is bounded is the ability to tell apart states *of one circuit*,

[2] The hash is a 16-byte `blake2b` digest. A collision would merge two groups that 1-WL separates and can only raise the reported value, so the bound would become conservative rather than optimistic; at 1,000 graphs per dataset the probability of one is negligible.

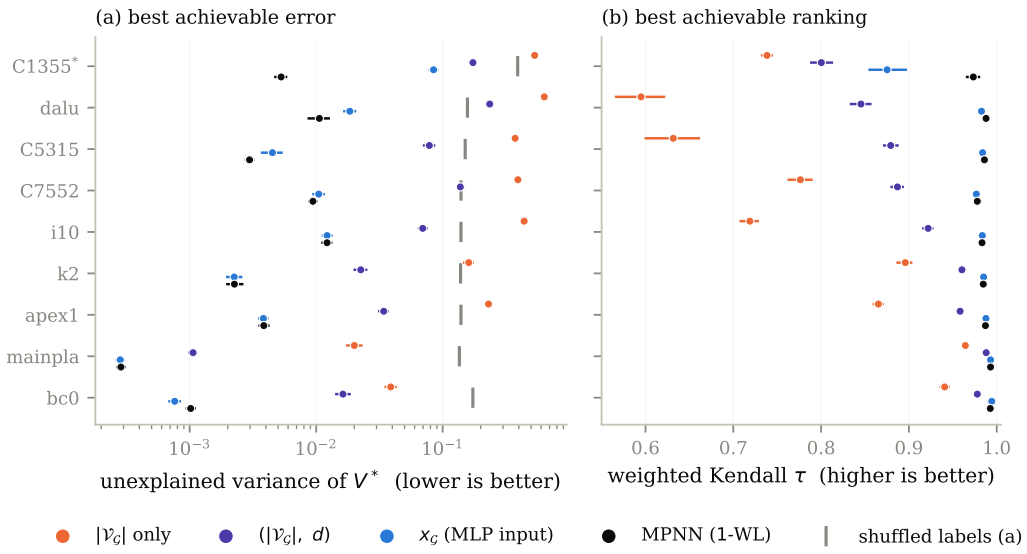


Figure 6: Best error (a) and best ranking (b) achievable by each model class on $\widehat{V}$, over nine benchmarks and ten dataset seeds. Each class is replaced by the optimal predictor for the information it can distinguish, so the value shown cannot be beaten by any model in that class. *`C1355` has only six distinct targets.

whereas the argument for learned encoders in Section 4.2 is transfer *across* circuits; our experiment does not test that claim, and a cross-circuit dataset could behave differently. Third, $\widehat{V}$ is a Monte-Carlo estimate, so part of its variance is sampling noise that no representation can predict. That noise raises every bound by roughly the same amount and therefore compresses the ratios between classes towards one, which works in favour of the negative conclusion we draw; the 1-WL and $x_\mathcal{G}$ bounds being close is evidence that message passing adds little, but not proof that the noise floor is not what they are both close to. Fourth, the 1-WL argument covers message passing networks whose input is the AIG with node-type features, which is the setting of Section 4.2 and of Zhu et al. [48]. It does not cover higher-order architectures, models given node identifiers or positional encodings, nor encoders whose node features are not a function of the graph alone—the simulation-derived features used by some AIG-specific models are an example, and for those the bound of Figure 6 is not binding.

5.2.3 Empirical performances of AIG-specific models on value prediction

The bounds of Section 5.2.2 say what a representation *could* express. This section asks what models actually trained on the same datasets achieve, which is the quantity a critic in Algorithm 1 would reach.

Models. We train eight regressors on the datasets of Section 5.2.1. Four are generic message passing networks—GCN [17], GIN [44], GAT [40] and GraphSAGE [13]—taken from `PyTorch Geometric` [8]. Three are AIG-specific: DeepGate [18], PolarGate [20] and FuncGNN [46], each run from its authors' implementation. The eighth is the MLP of Section 4.2.1, which reads $x_\mathcal{G}$ and no graph at all, and is the baseline the graph models have to beat. All eight share the same head so that only the encoder differs: node embeddings are mean-pooled into one vector per AIG, which a two-layer ReLU network maps to a scalar. GCN, GIN and GraphSAGE consume the six-way one-hot node type of Section 4.2; GAT, PolarGate and FuncGNN consume the three-way type (input, AND, inverter) of the DeepGate reader, with the inverter bit exposed as a signed edge feature, so that the models that were designed to use complementation receive it. DeepGate is the one exception to end-to-end training: we load the pretrained encoder, freeze it, and train only the head on its mean-pooled functional embeddings, which is how it is meant to be used as a general-purpose circuit encoder.

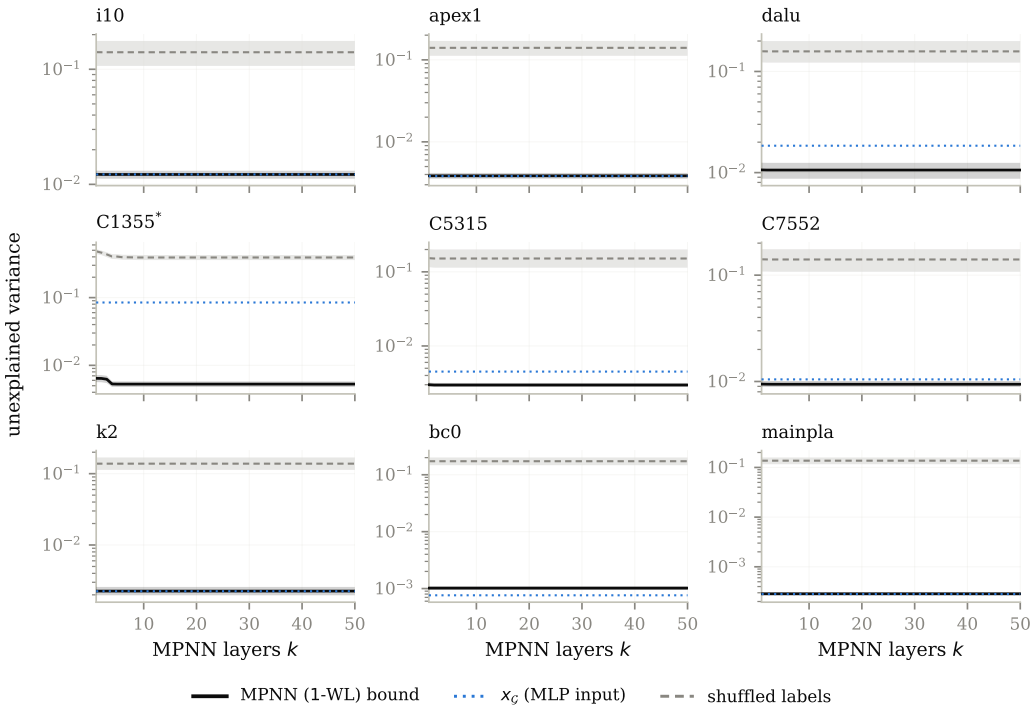


Figure 7: The MPNN bound as a function of depth k.

Training and selection. Targets are $\widehat{V}$ divided by the AND count of the benchmark's initial AIG, so they lie in $(0, 1]$ and are comparable across circuits. Each dataset of $1{,}000$ states is split $80/10/10$ into training, validation and test sets by a permutation seeded by the run seed. Models are trained for $1{,}000$ epochs with Adam [16] on the mean squared error, in mini-batches of 16 unless the batch size is searched over. Each architecture is given its own grid of 36 configurations—three learning rates (10^{-3}, 5×10^{-4}, 10^{-4}) crossed with a width and a depth, or with the number of attention heads for GAT and the polarity weight for PolarGate—and 12 for DeepGate, whose frozen encoder leaves only the head to tune. The grid is run at seed 0, the configuration with the lowest final validation error is selected, and that configuration alone is then rerun on the ten seeds we report; the seed selects the dataset, the split and the initialisation together, so the ten repetitions are independent in all three. We report test error normalised by the variance of the test targets, so that 1.0 is the error of predicting the mean.

Two caveats. The comparison is generous to the graph models in one respect and strict in another. It is generous because the MLP reads the full $x_{\mathcal{G}}$, action histogram included, whereas the $x_{\mathcal{G}}$ bound of Section 5.2.2 was computed without it: the baseline the graph models fail to beat here is stronger than the one they are compared against in Figure 6. It is strict because a single grid at seed 0, one head architecture and $1{,}000$ epochs is far less tuning than any of these encoders received in the papers that introduced them; we read the result below as evidence that they do not fit this target out of the box, not as an upper bound on what they could do with more effort.

Trained graph models, including AIG-specific ones, do not beat predicting the mean (Figures 8, 9). All seven graph models land between 0.90 and 1.74 times the target variance — at or above the constant predictor — while the MLP reaches 0.47 at the median. Their training error is also ≈ 1, so this is a failure to fit rather than to generalise, and it leaves them two to three orders of magnitude above the bound of Figure 6 that their own architecture class admits.

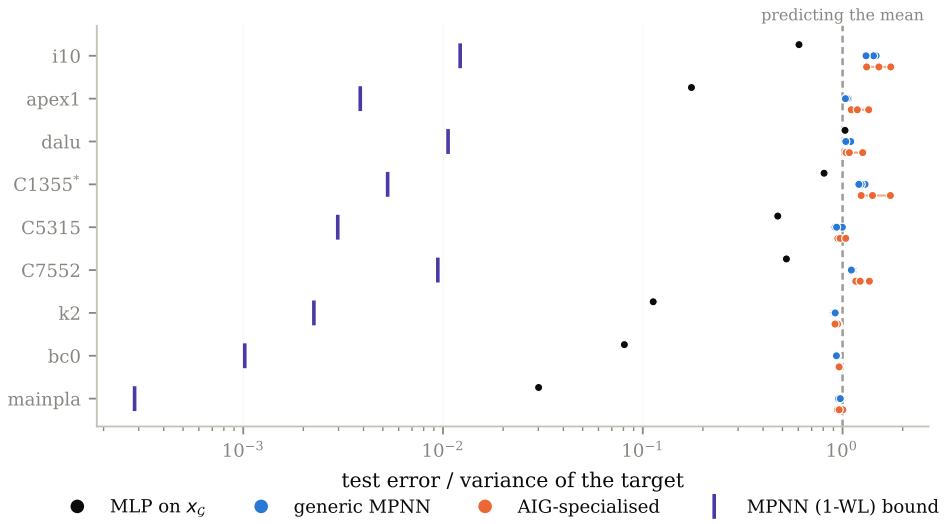


Figure 8: Test error of eight trained models, normalised by the variance of the target so that 1.0 is the error of predicting the mean. Violet ticks: the 1-WL bound of Figure 6.

6 Future work

Our results are restricted to the setting used by past work, in which the actions are the few `ABC` primitives of Section 2.1. An action space of that size gives the policy little to decide, and the states it can reach are the images of a handful of scripts, which may be why statistics suffice to tell them apart. A finer-grained action space—edits of individual nodes or cones rather than whole-graph scripts—would produce states that differ locally, and is the setting in which a structural representation has the most room to help. Measuring the bounds of Section 5.2 in that setting would say whether the negative result is a property of AIG reduction or of the action space it is usually given.

Two further directions follow from the material above. First, AIG editing is deterministic and its exploration problem is hard, a combination for which value-based algorithms [42, 25] are usually competitive, yet the existing work on AIG reduction is entirely policy gradient-based. Second, Stoll et al. [36] report that DAG generative models fail at AIG reduction without establishing why; the bounds we compute measure what a representation can distinguish, and applying them to the generative setting would test whether the same limitation is at play.

References

References

[1] Eren Akbiyik, Florian Grötschla, Béni Egressy, and Roger Wattenhofer. Graphtester: Exploring Theoretical Boundaries of GNNs on Graph Datasets. In *Data-centric Machine Learning Research (DMLR) Workshop at ICML 2023, Honolulu, Hawaii*, July 2023.

[2] Eric Allender and Bireswar Das. Zero knowledge and circuit minimization. *Information and Computation*, 256:2–8, 2017. ISSN 0890-5401. doi: https://doi.org/10.1016/j.ic.2017.04.004. URL `https://www.sciencedirect.com/science/article/pii/S0890540117300512`.

[3] Richard Bellman. *Dynamic Programming*. 1957.

[4] Robert K. Brayton and Alan Mishchenko. Abc: An academic industrial-strength verification tool. In *International Conference on Computer Aided Verification*, 2010. URL `https://api.semanticscholar.org/CorpusID:1251520`.

[5] Alba Carballo-Castro, Manuel Madeira, Yiming QIN, Dorina Thanou, and Pascal Frossard. Generating directed graphs with dual attention and asymmetric encoding. In *The Fourteenth*

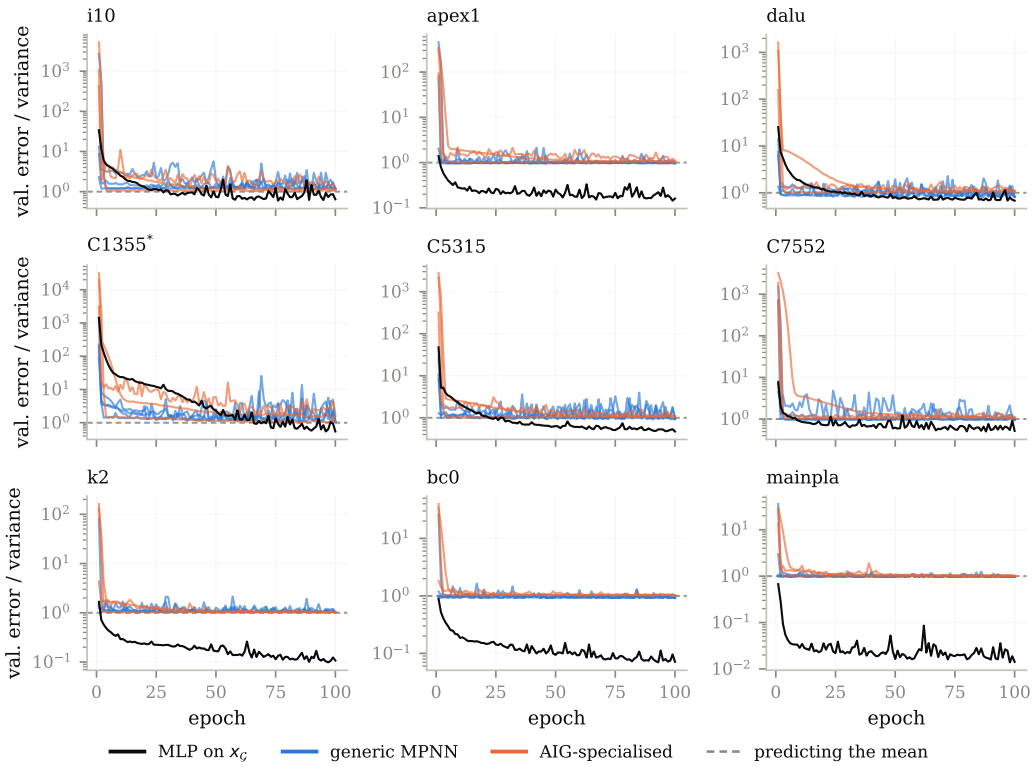


Figure 9: Validation error over training for the configurations of Figure 8.

International Conference on Learning Representations, 2026. URL `https://openreview.net/forum?id=s2s5xGQCKM`.

[6] Andrea Costamagna, Alan Mishchenko, Satrajit Chatterjee, and Giovanni De Micheli. An enhanced resubstitution algorithm for area-oriented logic optimization. In *2024 IEEE International Symposium on Circuits and Systems (ISCAS)*, pages 1–5, 2024. doi: 10.1109/ISCAS58744.2024.10558264.

[7] Victor-Alexandru Darvariu, Stephen Hailes, and Mirco Musolesi. Graph reinforcement learning for combinatorial optimization: A survey and unifying perspective. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL `https://openreview.net/forum?id=HduK51xNtS`. Survey Certification.

[8] Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with pytorch geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[9] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1263–1272. PMLR, 2017.

[10] Antoine Grosnit, Cedric Malherbe, Rasul Tutunov, Xingchen Wan, Jun Wang, and Haitham Bou Ammar. Boils: Bayesian optimisation for logic synthesis. In *Design, Automation, Test in Europe Conference, 2022*, pages 1193–1196, 2022. doi: 10.23919/DATE54114.2022.9774632.

[11] Winston Haaswijk, Edo Collins, Benoit Seguin, Mathias Soeken, Frédéric Kaplan, Sabine Süsstrunk, and Giovanni De Micheli. Deep learning for logic optimization algorithms. In *2018 IEEE International Symposium on Circuits and Systems (ISCAS)*, pages 1–4, 2018. doi: 10.1109/ISCAS.2018.8351885.

[12] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using networkx. In *Proceedings of the 7th Python in Science Conference*, pages 11–15, 2008.

[13] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[14] Abdelrahman Hosny, Soheil Hashemi, Mohamed Shalan, and Sherief Reda. Drills: Deep reinforcement learning for logic synthesis. In *2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 581–586, 2020. doi: 10.1109/ASP-DAC47756.2020.9045559.

[15] Valentine Kabanets and Jin-Yi Cai. Circuit minimization problem. In *Proceedings of the thirty-second annual ACM symposium on Theory of computing*, pages 73–79, 2000.

[16] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.

[17] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017.

[18] Min Li, Sadaf Khan, Zhengyuan Shi, Naixing Wang, Huang Yu, and Qiang Xu. Deepgate: Learning neural representations of logic gates. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*, pages 667–672, 2022.

[19] Mufei Li, Viraj Shitole, Eli Chien, Changhai Man, Zhaodong Wang, Srinivas, Ying Zhang, Tushar Krishna, and Pan Li. LayerDAG: A layerwise autoregressive diffusion model of directed acyclic graphs for system. In *Machine Learning for Computer Architecture and Systems 2024*, 2024. URL `https://openreview.net/forum?id=IsarrieeQA`.

[20] Jiawei Liu, Jianwang Zhai, Mingyu Zhao, Zhe Lin, Bei Yu, and Chuan Shi. Polargate: Breaking the functionality representation bottleneck of and-inverter graph neural network. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, 2024.

[21] Azalia Mirhoseini, Anna Goldie, Mustafa Yazgan, Joe Wenjie Jiang, Ebrahim M. Songhori, Shen Wang, Young-Joon Lee, Eric Johnson, Omkar Pathak, Azade Nazi, Jiwoo Pak, Andy Tong, Kavya Srinivasa, Will Hang, Emre Tuncer, Quoc V. Le, James Laudon, Richard Ho, Roger Carpenter, and Jeff Dean. A graph placement methodology for fast chip design. *Nature*, 594: 207 – 212, 2021. URL `https://api.semanticscholar.org/CorpusID:235395490`.

[22] A. Mishchenko, S. Chatterjee, and R. Brayton. Dag-aware aig rewriting: a fresh look at combinational logic synthesis. In *2006 43rd ACM/IEEE Design Automation Conference*, pages 532–535, 2006. doi: 10.1145/1146909.1147048.

[23] Alan Mishchenko and Robert K. Brayton. Scalable logic synthesis using a simple circuit structure. 2006. URL `https://api.semanticscholar.org/CorpusID:8597391`.

[24] Alan Mishchenko, Satrajit Chatterjee, Roland Jiang, and Robert K. Brayton. Fraigs: A unifying representation for logic synthesis and verification, 2005. URL `https://api.semanticscholar.org/CorpusID:9209313`.

[25] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533, 2015.

[26] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In Maria Florina Balcan and Kilian Q. Weinberger, editors, *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 1928–1937, New York, New York, USA, 20–22 Jun 2016. PMLR. URL `https://proceedings.mlr.press/v48/mniha16.html`.

[27] Christopher Morris, Martin Ritzert, Matthias Fey, William L. Hamilton, Jan Eric Lenssen, Gaurav Rattan, and Martin Grohe. Weisfeiler and leman go neural: Higher-order graph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 4602–4609, 2019.

[28] Walter Lau Neto, Yingjie Li, Pierre-Emmanuel Gaillardon, and Cunxi Yu. Flowtune: End-to-end automatic logic optimization exploration via domain-specific multiarmed bandit. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 42(6):1912–1925, 2023. doi: 10.1109/TCAD.2022.3213611.

[29] Suhua Ou, Qingshan Yang, and Jian Liu. The global production pattern of the semiconductor industry: an empirical research based on trade network. *Humanities and Social Sciences Communications*, 11(1):1–13, 2024.

[30] Rajendran Panda and Farid N. Najm. Post-mapping transformations for low-power synthesis. *VLSI Design*, 7:289–301, 1998. URL `https://api.semanticscholar.org/CorpusID:6811282`.

[31] Zehua Pei, Fangzhou Liu, Zhuolun He, Guojin Chen, Haisheng Zheng, Keren Zhu, and Bei Yu. Alphasyn: Logic synthesis optimization with efficient monte carlo tree search. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, pages 1–9, 2023. doi: 10.1109/ICCAD57390.2023.10323856.

[32] Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, 1994.

[33] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.

[34] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[35] Chuan Shi, Yitong Li, Jiawei Zhang, Yizhou Sun, and Philip S Yu. A survey of heterogeneous information network analysis. *IEEE Transactions on Knowledge and Data Engineering*, 29(1): 17–37, 2016.

[36] Timo Stoll, Chendi Qian, Ben Finkelshtein, Ali Parviz, Darius Weber, Fabrizio Frasca, Hadar Shavit, Antoine Siraudin, Arman Mielke, Marie Anastacio, Erik Müller, Maya Bechler-Speicher, Michael Bronstein, Mikhail Galkin, Holger Hoos, Mathias Niepert, Bryan Perozzi, Jan Tönshoff, and Christopher Morris. Graphbench: Next-generation graph learning benchmarking, 2026. URL `https://arxiv.org/abs/2512.04475`.

[37] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. A Bradford Book, Cambridge, MA, USA, 2018. ISBN 0262039249.

[38] Richard S Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In S. Solla, T. Leen, and K. Müller, editors, *Advances in Neural Information Processing Systems*, volume 12. MIT Press, 1999. URL `https://proceedings.neurips.cc/paper_files/paper/1999/file/464d828b85b0bed98e80ade0a5c43b0f-Paper.pdf`.

[39] Gerald Tesauro. Temporal difference learning and td-gammon. *Commun. ACM*, 38(3):58–68, March 1995. ISSN 0001-0782. doi: 10.1145/203330.203343. URL `https://doi.org/10.1145/203330.203343`.

[40] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations*, 2018.

[41] Hanrui Wang, Kuan Wang, Jiacheng Yang, Linxiao Shen, Nan Sun, Hae-Seung Lee, and Song Han. Gcn-rl circuit designer: Transferable transistor sizing with graph neural networks and reinforcement learning. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6, 2020. doi: 10.1109/DAC18072.2020.9218757.

[42] Christopher JCH Watkins and Peter Dayan. Q-learning. *Machine learning*, 8(3):279–292, 1992.

[43] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Mach. Learn.*, 8(3–4):229–256, May 1992. ISSN 0885-6125. doi: 10.1007/BF00992696. URL https://doi.org/10.1007/BF00992696.

[44] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations*, 2019.

[45] Wenlong Yang, Lingli Wang, and Alan Mishchenko. Lazy man's logic synthesis. In *Proceedings of the International Conference on Computer-Aided Design*, ICCAD '12, page 597–604, New York, NY, USA, 2012. Association for Computing Machinery. ISBN 9781450315739. doi: 10.1145/2429384.2429513. URL https://doi.org/10.1145/2429384.2429513.

[46] Qiyun Zhao. Funcgnn: Learning functional semantics of logic circuits with graph neural networks, 2025.

[47] Ziyang Zheng, Shan Huang, Jianyuan Zhong, Zhengyuan Shi, Guohao Dai, Ningyi Xu, and Qiang Xu. Deepgate4: Efficient and effective representation learning for circuit design at scale. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=b10lRabU9W.

[48] Keren Zhu, Mingjie Liu, Hao Chen, Zheng Zhao, and David Z. Pan. Exploring logic optimizations with reinforcement learning and graph convolutional network. In *2020 ACM/IEEE 2nd Workshop on Machine Learning for CAD (MLCAD)*, pages 145–150, 2020. doi: 10.1145/3380446.3430622.

A Technical appendices and supplementary material

Technical appendices with additional results, figures, graphs, and proofs may be submitted with the paper submission before the full submission deadline (see above). You can upload a ZIP file for videos or code, but do not upload a separate PDF file for the appendix. There is no page limit for the technical appendices.

Note: Think of the appendix as "optional reading" for reviewers. The paper must be able to stand alone without the appendix; for example, adding critical experiments that support the main claims to an appendix is inappropriate.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive" or "we were unable to find the license for the dataset we used"). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading "NeurIPS Paper Checklist"**,
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers**.

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: **[TODO]**

   Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.

- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).

- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
   - For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
   - If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.

- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
    - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.

- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: **[TODO]**

    Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.